Advanced computerarchitecture
2D fluid Dynamics

(B-KUL-JPI30R)

Wout Van Steenbergen, Timon Coen

May 9, 2024



Instructor: Alex Marinšek

1 Introduction

This report examines the functioning of a 2D fluid simulation, built on the basic techniques presented by Jos Stam in his seminal paper on real-time fluid dynamics for games [1]. It begins with a detailed overview of the dynamics solver's operation, outlining the mathematical and computational principles that enable the simulation of fluid motion. The discussion extends to a selection of programming languages. Highlighting their respective strengths and limitations in relation to fluid dynamics modeling.

Furthermore, the paper explores the integration of CUDA to significantly enhance the simulation's performance. By leveraging CUDA, both computational calculations and graphical rendering can be accelerated, thereby achieving higher framerates and more responsive simulation experiences. In addition to these core improvements, the paper also examines a variety of supplementary features and optimizations that have been examined to further speed up the simulation.

2 Fluid Dynamics

Fluid dynamics is a branch of physics concerned with the study of fluids (liquids, gases, and plasmas) and the forces on them. It is a critical area of research in both engineering and science, encompassing various applications from designing aircraft and cars to understanding the flow of blood in the human body. This field is characterized by its complex mathematical models which predict how fluids behave under various conditions.

In computational fluid dynamics (CFD), models are solved using numerical approaches, making it possible to simulate fluid interaction with surfaces, and to visualize flow patterns that are otherwise invisible. The equations at the heart of fluid dynamics are the Navier-Stokes equations, which describe the motion of fluid substances. These equations are nonlinear and typically impossible to solve analytically except for the simplest cases. Therefore, CFD relies heavily on computing power to create simulations that predict fluid behavior in more complex scenarios.

Due to the significant computational demands required to accurately model fluid dynamics, most systems are unable to perform these calculations effectively. As a result, a simplified implementation is proposed by Jos Stam [1]. This implementation is designed to be faster and less resource-intensive, while still closely approximating the appearance and behavior of real fluid dynamics.

2.1 Navier-Stokes Equations

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u}\right) = -\nabla p + \mu\nabla^2\mathbf{u} + \mathbf{f} \tag{1}$$

In simulating scenarios like smoke from a cigarette or dust in the air, modeling each individual particle becomes impractical due to the massive number of particles and their complex interactions. Instead, using a density field approach Eq. 2 simplifies the process. This method treats the particles as a continuous substance, focusing on the overall movement and concentration rather than the position of each particle. This approach not only reduces the need for computational power but also simplifies the mathematical treatment. While the equations governing fluid velocity (Navier-Stokes equations) Eq. 1 are non-linear and complex, the equations for the evolution of density Eq. 2 in a fluid are linear. This linearity makes the mathematical processes less complex [1].

$$\underbrace{\frac{\partial \rho}{\partial t}}_{\text{over time}} = \underbrace{-(\mathbf{u}\cdot\nabla)\rho}_{\text{advection}} + \underbrace{\kappa\nabla^2\rho}_{\text{diffusion}} + \underbrace{S}_{\text{external force}} \tag{2}$$

This formula can be interpreted and explained in a straightforward manner, progressing from left to right. Initially, the formula indicates that the pressure within a system changes over time due to several influencing factors, which are outlined on the right side of the equation. The first of these factors is **advection**, a process where the velocity of a fluid carries itself along with other quantities within the fluid. Following advection, there's a term representing the **diffusion** of density, which is modulated by a constant K. This process is linked to **viscosity**, which describes the fluid's resistance to flow. The final term in the equation accounts for the **external addition** of density into the system.

2.2 Implementation

Jos Stam [1] describes an implementation that is constrained to two dimensions. The fluid is modeled on a square grid, similar to the one illustrated in Figure 1. To simplify the treatment of boundary conditions, an additional layer of grid cells is added around the fluid's domain. It is assumed that both the velocity and the density of the fluid remain constant within each grid cell, and their values are typically displayed at the center of the cell. For practical purposes, arrays are used to store data: one array for density and two for velocity (reflecting the u and v components of velocity vectors), with each array having a size of (N+2)*(N+2) [1].

Single-dimensional arrays are preferred over multi-dimensional ones for efficiency. Elements within these arrays are accessed using the macro defined as: #define IX(i,j) ((i)+(N+2)*(j)) [1].

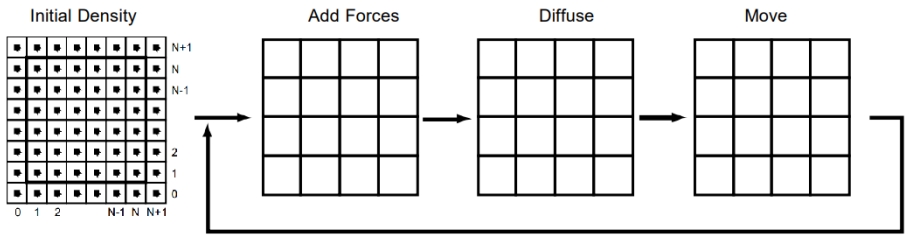


Figure 1: Density solver with the Initial Density presenting the Computational grid [1].

2.2.1 Density solver

First, a solver is designed for a density field that moves through a fixed velocity field which does not change over time. Consider the density equation Eq. 2 shown above. This equation indicates that changes in density during a single time step are caused by three factors. These are represented by the terms on the right-hand side of the equation. The solver tackles these terms in the reverse order of their appearance, as illustrated in Figure 1. Starting with an initial density, these three terms are resolved in each subsequent time step. The first term is straightforward to implement. It is assumed that sources for a given frame are stored in an array, which, in the simulation, is populated based on user's mouse movements. The routine that integrates the source into the density simply adds the values from this array.

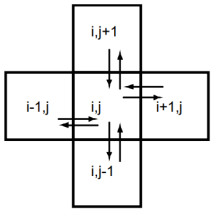


Figure 2: Density exchange with neighbors in single grid cell [1].

The second step involves accounting for diffusion at a rate specified by diff. When diff $>$ 0, the density will spread across the grid cells. To better understand this, consider what occurs in a single grid cell. It is assumed that the cell exchanges densities only with its four direct neighbors, as illustrated in Figure 2. The density within the cell decreases as it loses density to its neighbors but also increases as densities flow in from those same neighbors. This exchange leads to a net difference in the cell's density, which is calculated as follows[1]:

$$x[\mathrm{IX}(i,j)] = x_0[\mathrm{IX}(i,j)] + a(x_0[\mathrm{IX}(i-1,j)] + x_0[\mathrm{IX}(i+1,j)] + x_0[\mathrm{IX}(i,j-1)] \\ + x_0[\mathrm{IX}(i,j+1)] - 4x_0[\mathrm{IX}(i,j)])$$

$$a = dt \cdot \mathrm{diff} \cdot N^2$$

A big problem occurs for large diffusion rates, they often lead to oscillations, negative values, and divergence in density calculations, rendering the simulation ineffective. To address this, the Gauss-Seidel relaxation technique, a stable diffusion method, has been adopted. It calculates densities such that, if diffused backward in time, they would revert to their original state[1].

$$x[\mathrm{IX}(i,j)] = \frac{x_0[\mathrm{IX}(i,j)] + a\left(x[\mathrm{IX}(i-1,j)] + x[\mathrm{IX}(i+1,j)] + x[\mathrm{IX}(i,j-1)] + x[\mathrm{IX}(i,j+1)]\right)}{1+4a}$$

The Gauss-Seidel relaxation method is an iterative technique used to solve linear equations. It updates the solution approximation sequentially for each equation, using the latest values as soon as they are available. This approach speeds up convergence over methods that only update the solution after all equations are considered. In the context of this simulation, it ensures stability and efficiency, allowing the system to handle any size of diffusion rate, time step, or grid dimension without the risk of blowing up [2].

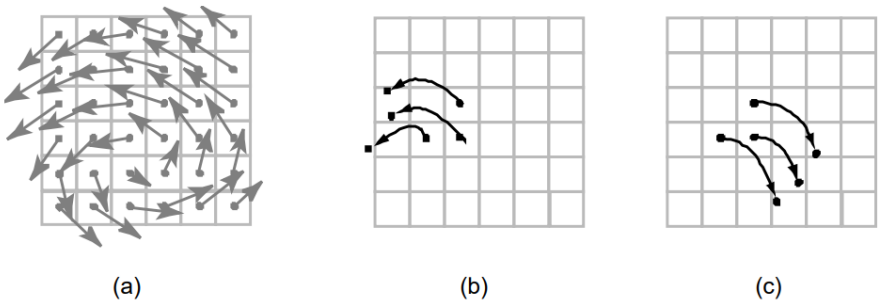


Figure 3: Velocity field (a), moving cell the centers forward (b), tracing cell centers backwards (c) [1].

The final step in the density solver is to force the density to follow a given velocity field. Instead of solving complex linear systems influenced by velocity, treat the density as if it were particles moving in the flow. Assume each grid cell's center is a particle traced through the velocity field as shown in Figure 3 (b). Instead of converting these movements back to grid values directly, follow each grid point backwards through the flow to see where it came from as shown in Figure 3 (C). Then, use the densities from the closest four points of its original position to estimate its new density. Use two grids for this process: one to hold the old densities and one for the updated densities. This method is simpler and avoids complicated calculations [1].

Algorithm 1 Advect

Require: grid size N, boundary type b, density array d, previous density array d_0, velocity fields u and v, time step dt

Ensure: updated density field d influenced by velocity fields

Compute scaled time step $dt_0 = dt \times N$
for $i = 1$ to N **do**
  for $j = 1$ to N **do**
    Calculate backward trace for x and y:
    $x \leftarrow i - dt_0 \times u[\text{IX}(i, j)]$
    $y \leftarrow j - dt_0 \times v[\text{IX}(i, j)]$
    Clamp x and y to prevent escaping grid boundaries:
    $x \leftarrow \max(\min(x, N + 0.5), 0.5)$
    $y \leftarrow \max(\min(y, N + 0.5), 0.5)$
    Determine integer grid indices:
    $i_0 \leftarrow \lfloor x \rfloor$, $j_0 \leftarrow \lfloor y \rfloor$
    $i_1 \leftarrow i_0 + 1$,$j_1 \leftarrow j_0 + 1$
    Calculate interpolation weights:
    $s_1 \leftarrow x - i_0$, $s_0 \leftarrow 1 - s_1$
    $t_1 \leftarrow y - j_0$, $t_0 \leftarrow 1 - t_1$
    Interpolate density:
    $d[\text{IX}(i, j)] \leftarrow s_0 \times (t_0 \times d_0[\text{IX}(i_0, j_0)] + t_1 \times d_0[\text{IX}(i_0, j_1)]) + s_1 \times (t_0 \times d_0[\text{IX}(i_1, j_0)] + t_1 \times d_0[\text{IX}(i_1, j_1)])$
  end for
end for
Apply boundary conditions:
`set_bnd`(N, b, d)

This completes the density solver. All of these steps can conveniently be grouped together into a single routine. SWAP is a macro that swaps the two array pointers.

Algorithm 2 Density Step

Require: grid size N, current density field x, previous density field x_0, velocity fields u and v, diffusion coefficient $diff$, time step dt

Ensure: updated density field x after applying source addition, diffusion, and advection

1:	`add_source`(N, x, x_0, dt)	Add sources to density
2:	`SWAP`(x_0, x)	Swap x_0 and x for diffusion step
3:	`diffuse`(N, 0, x, x_0, $diff$, dt)	*Diffuse the density*
4:	`SWAP`(x_0, x)	Swap x_0 and x for advection step
5:	`advect`(N, 0, x, x_0, u, v, dt)	Advect the density using velocity fields

2.2.2 Velocity solver

Jos Stam's paper [1] now continues with the implemetation of the velocity solver. But now the velocity equation Eq. 1 needs to be used. By reformulating the equation, the same logic as before can be used. It says that the velocity changes over a time span due to addition of forces, viscous diffusion and self-advection. Although the term "self-advection" might sound odd, it simply means that the velocity field moves along its own flow. The routines developed for density can be efficiently reused for this purpose.

Algorithm 3 Velocity Step

Require: grid size N, current velocity fields u, v, previous velocity fields u_0, v_0, viscosity $visc$, time step dt

Ensure: updated velocity fields u and v after applying source addition, diffusion, advection, and projection

1:	`add_source`(N, u, u_0, dt)	Add sources to u
2:	`add_source`(N, v, v_0, dt)	Add sources to v
3:	`SWAP`(u_0, u)	Swap u_0 and u for diffusion step
4:	`diffuse`(N, 1, u, u_0, $visc$, dt)	Diffuse u
5:	`SWAP`(v_0, v)	Swap v_0 and v for diffusion step
6:	`diffuse`(N, 2, v, v_0, $visc$, dt)	Diffuse v
7:	`project`(N, u, v, u_0, v_0)	Project to make divergence free
8:	`SWAP`(u_0, u)	Prepare u for advection
9:	`SWAP`(v_0, v)	Prepare v for advection
10:	`advect`(N, 1, u, u_0, u_0, v_0, dt)	Advect u using velocity fields
11:	`advect`(N, 2, v, v_0, u_0, v_0, dt)	Advect v using velocity fields
12:	`project`(N, u, v, u_0, v_0)	Final project to ensure divergence free

There is an addition in the form of a new routine called project() that is not in the density solver. This routine ensures the velocity is mass conserving. Essentially, every velocity field consists of an incompressible field and a gradient field (Figure 4 top). To achieve an incompressible field, the gradient field is subtracted from the current velocities (Figure 4 bottom). This subtraction is crucial as it mimics a key property of real fluids—maintaining incompressibility. Visually, this enforcement generates numerous vortices, producing flows with realistic, swirly patterns. The project() routine is called twice because the advect() routine behaves better for a mass conserving velocity field [1].

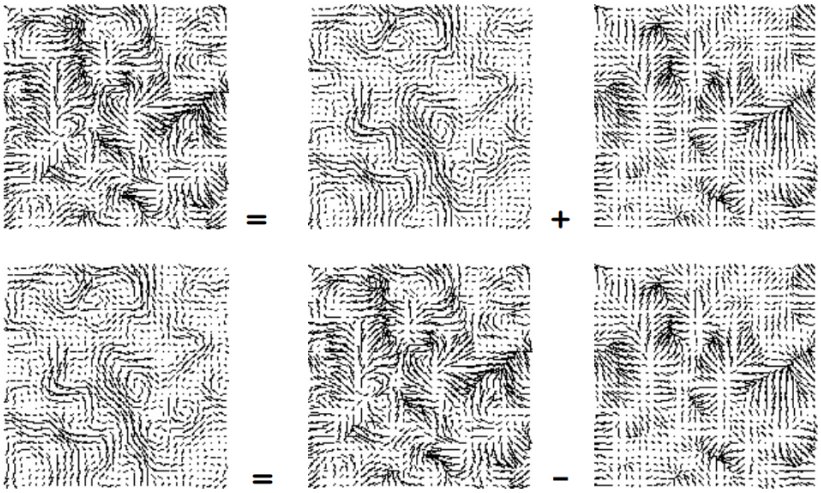


Figure 4: Components of velocity field [1].

Algorithm 4 Project

Require: velocity fields u, v, pressure field p, divergence field div, grid size N
Ensure: updated velocity fields u, v that are divergence free

Initialize divergence and pressure:
for $i = 1$ to N **do**
  for $j = 1$ to N **do**
    Calculate divergence at each cell:
    $div[\text{IX}(i,j)] \leftarrow -0.5 \times (u[\text{IX}(i+1,j)] - u[\text{IX}(i-1,j)] + v[\text{IX}(i,j+1)] - v[\text{IX}(i,j-1)])/N$
    Initialize pressure at each cell to zero:
    $p[\text{IX}(i,j)] \leftarrow 0$
  end for
end for
Apply boundary conditions to divergence and pressure:
`set_bnd`(0, div)
`set_bnd`(0, p)
Solve for pressure:
`lin_solve`(0, p, div, 1, 4)
Adjust velocities to ensure non-divergence:
for $i = 1$ to N **do**
  for $j = 1$ to N **do**
    Adjust u velocity component:
    $u[\text{IX}(i,j)] \leftarrow u[\text{IX}(i,j)] - 0.5 \times N \times (p[\text{IX}(i+1,j)] - p[\text{IX}(i-1,j)])$
    Adjust v velocity component:
    $v[\text{IX}(i,j)] \leftarrow v[\text{IX}(i,j)] - 0.5 \times N \times (p[\text{IX}(i,j+1)] - p[\text{IX}(i,j-1)])$
  end for
end for
Reapply boundary conditions to adjusted velocity fields:
`set_bnd`(1, u)
`set_bnd`(2, v)

The treatment of the boundary in the solver is addressed by the set_bnd() routine, which is frequently used throughout the code. This routine assumes that the fluid is contained within a box with solid walls, ensuring that no flow exits the walls. Specifically, this means that the horizontal component of the velocity must be zero at the vertical walls, and the vertical component of the velocity must be zero at the horizontal walls. For the density and other fields considered in the code, continuity is assumed, allowing for uninterrupted flow characteristics across these boundaries [1].

Algorithm 5 Set Boundary Conditions

Require: grid size N, boundary type b, array x
Ensure: boundary conditions are applied to array x
for $i = 1$ to N **do**
    Set left boundary:
    $x[\text{IX}(0, i)] \leftarrow (b == 1)? - x[\text{IX}(1, i)] : x[\text{IX}(1, i)]$
    Set right boundary:
    $x[\text{IX}(N + 1, i)] \leftarrow (b == 1)? - x[\text{IX}(N, i)] : x[\text{IX}(N, i)]$
    Set bottom boundary:
    $x[\text{IX}(i, 0)] \leftarrow (b == 2)? - x[\text{IX}(i, 1)] : x[\text{IX}(i, 1)]$
    Set top boundary:
    $x[\text{IX}(i, N + 1)] \leftarrow (b == 2)? - x[\text{IX}(i, N)] : x[\text{IX}(i, N)]$
end for
Adjust corner cells to be the average of their two adjacent cells:
$x[\text{IX}(0, 0)] \leftarrow 0.5 \times (x[\text{IX}(1, 0)] + x[\text{IX}(0, 1)])$
$x[\text{IX}(0, N + 1)] \leftarrow 0.5 \times (x[\text{IX}(1, N + 1)] + x[\text{IX}(0, N)])$
$x[\text{IX}(N + 1, 0)] \leftarrow 0.5 \times (x[\text{IX}(N, 0)] + x[\text{IX}(N + 1, 1)])$
$x[\text{IX}(N + 1, N + 1)] \leftarrow 0.5 \times (x[\text{IX}(N, N + 1)] + x[\text{IX}(N + 1, N)])$

To make the **simulation implementation** complete, the velocity and density step are put together with the UI input and drawing routine in a 'while' loop.

Algorithm 6 Fluid Dynamics Simulation Loop

Require: grid size N, viscosity $visc$, diffusion coefficient $diff$, time step dt
Ensure: Updated fluid dynamics system visualized per iteration
while simulating **do**
    `get_from_UI`($dens_prev$, u_prev, v_prev)
    `vel_step`(N, u, v, u_prev, v_prev, $visc$, dt)
    `dens_step`(N, $dens$, $dens_prev$, u, v, $diff$, dt)
    `draw_dens`(N, $dens$)
end while

3 Programming language

CUDA can be employed to enhance the fluid simulation performance in both Python and C++. In Python, the advantages include easier integration and higher-level libraries, which significantly simplify development. However, achieving similar performance gains in C++ is more challenging and necessitates extensive coding. To evaluate the performance and coding efficiency, implementations based on Stam's paper [1] were developed in these languages. Each simulation was executed on the CPU using a grid of 64x64 vector points and a display resolution of 1024px by 1024px. In both programs, the diffusion and viscosity were set to zero, with a time step (dt) of 0,1. Although the C++ implementation required 24% more code than the Python counterpart, it delivered performance that was, on average, more than fourteen times faster. As a result, it was decided to implement CUDA in C++.

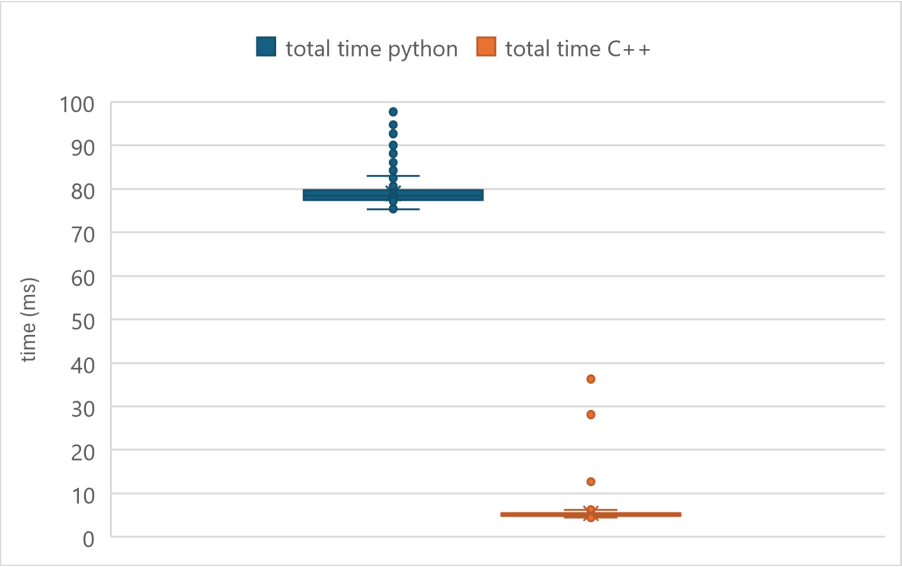


Figure 5: Python vs C++ calculation time of one frame (1K data points).

4 Cuda

Cuda technology can substantially enhance the performance of Jos Stam's fluid dynamics implementation. In many of his routines, each cell within the density or velocity arrays is iterated over by two nested for-loops, leading to a time complexity of N^2. This approach, while straightforward, is computationally intensive and not optimal for performance.

Algorithm 7 Standard CPU Function

Require: Argument arg, grid size N

for $i = 1$ to N **do**
    for $j = 1$ to N **do**
        Perform the specified routine on arg at position (i, j)
    end for
end for

Instead of processing each cell sequentially, Cuda allows for parallel processing of these cells. By leveraging Cuda's capability to handle multiple calculations simultaneously, significant improvements in processing speed can be achieved. This change requires rewriting the existing code to adapt to the parallel processing architecture of Cuda.

Algorithm 8 GPU Kernel Function

Require: Argument arg, grid size N including boundaries $(N + 2) \times (N + 2)$

Calculate 1D index from thread and block IDs:
$index = threadIdx.x + blockIdx.x \times blockDim.x$
Convert 1D index to 2D indices:
$i = index\%(N + 2)$
$j = index/(N + 2)$
if $i \geq 1$ and $i \leq N$ and $j \geq 1$ and $j \leq N$ **then**
    Perform the specified routine on arg at position (i, j)
end if

The same method can also be applied to calculate the color intensity for each pixel. This is achieved by taking the average density from its right, bottom and right-bottom neighbors, translating this average into a color intensity. However, the actual rendering process with SFML is not accelerated through parallelization on the GPU.

5 Results

To conduct a thorough assessment of performance enhancements, both the CPU and GPU implementations were subjected to a simulation involving a grid of 512x512 vector points, with the visualization displayed on a window resolution of 1024x1024 pixels (Figure 8). For this test, both implementations were set to have diffusion and viscosity parameters at zero, and a time step (dt) of 0.05 was used.

Figure 6 provides a clear visual comparison of the computational times for each implementation. Utilizing the GPU resulted in a dramatic acceleration, with calculations being over 20 times faster on average compared to the CPU implementation. This substantial speed-up demonstrates the GPU's robust performance in handling intensive computational tasks without the delays typically associated with CPU processing.

However, it's important to consider the overall system performance. Figure 7 expands on this by showing the total operational time, which includes both computation and the time spent on input handling and drawing. In these results, the improvement in speed is around 10 times. This indicates that while the GPU excels in computation, the advantages are somewhat mitigated by the unchanged durations required for input processing and the minimal gains in drawing speeds with CUDA.

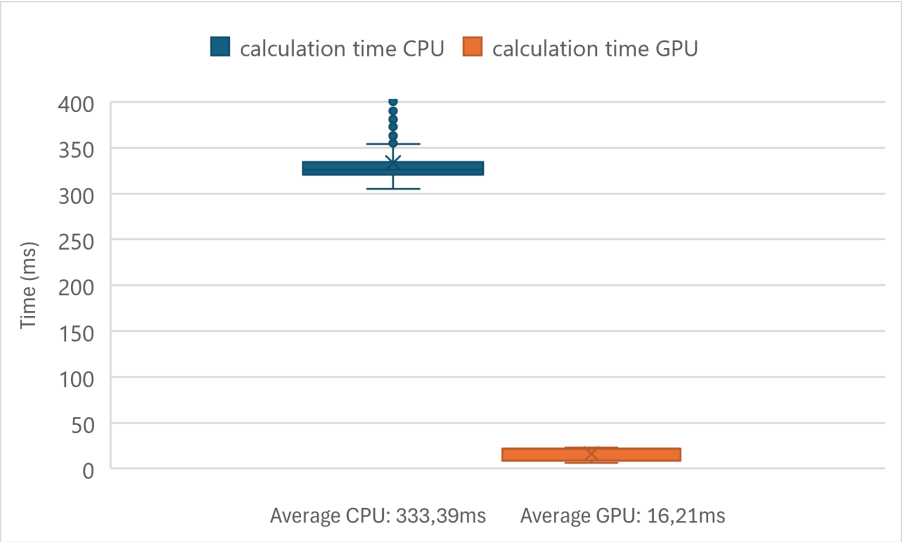


Figure 6: CPU vs GPU calculation time of one frame (1K data points).

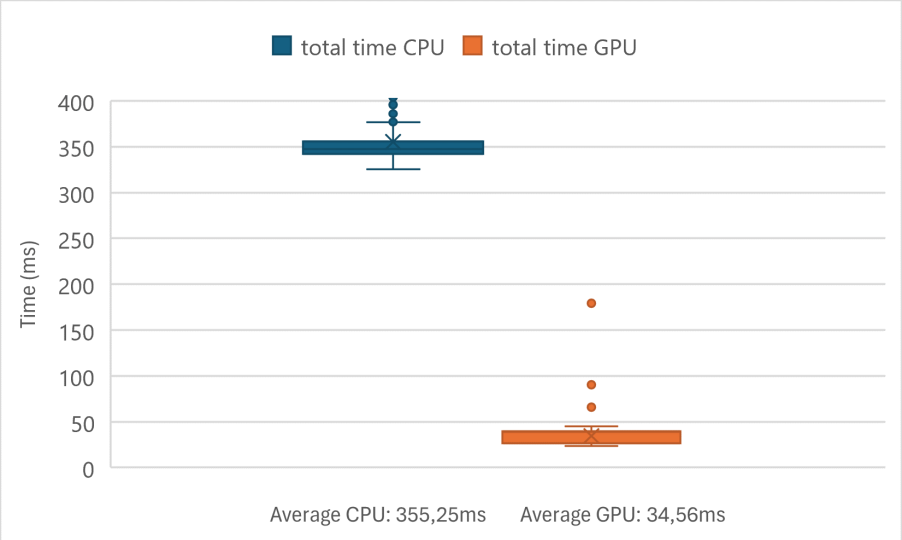


Figure 7: CPU vs GPU total time of one frame (1K data points).



Figure 8: Screenshot of simulation.

5.1 Coalesced memory access

Coalescing involves grouping memory accesses by threads in a way that they can fetch data in a single, aligned transaction. This technique significantly reduces the latency associated with accessing device memory by minimizing the number of memory transactions.

In the current implementation, coalesced memory requests are made during various routines. For example, in the linear solver where the values of neighboring grid cells are requested, the ones on the left and right can be accessed simultaneously through coalescing, thus eliminating the need for other threads to make requests to slow device memory. For the bottom and top neighbors, this is more challenging because they are stored further apart in the 1D array. However, all operations of all threads are executed simultaneously through warp execution, where a warp consists of a fixed number of threads (typically 32) that execute instructions in lockstep as shown in Figure 9 the green block. This allows these segments of memory to be fetched together, enabling coalescing.

To substantiate this claim, we have altered the implementation of CUDA so that the threads of a block are now divided vertically instead of horizontally, shown in Figure 9 in the red block. As a result, coalescing is no longer possible, and it is clearly demonstrated in Figure 10 that the computations are approximately twice as slow.

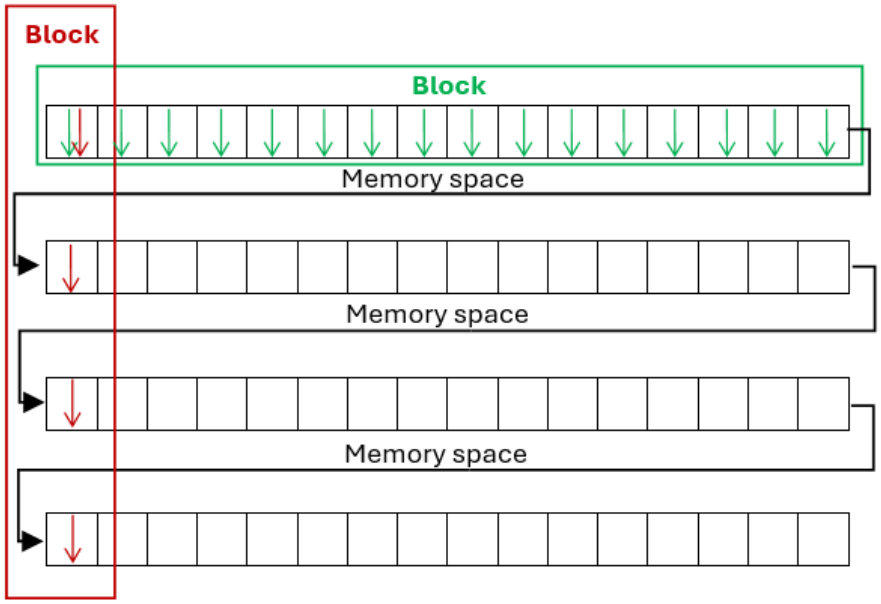


Figure 9: Different thread implementation for coalesced and un-coalesced memory access.

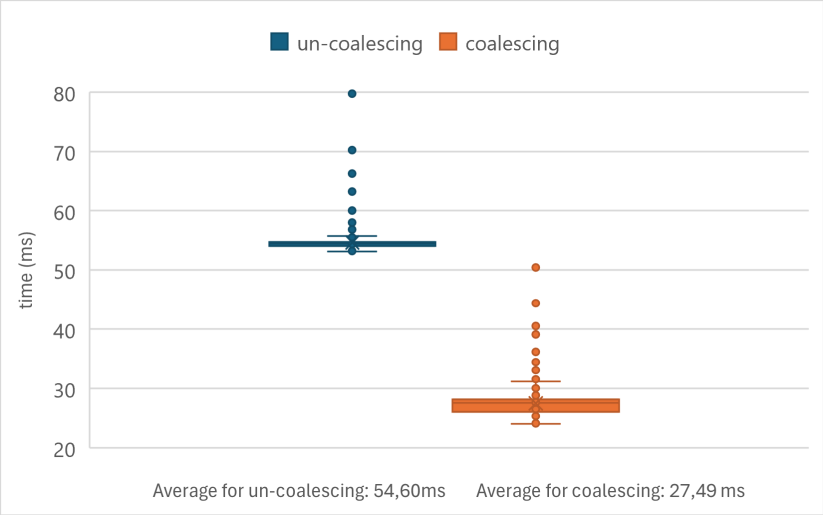


Figure 10: Un-coalescing vs coalescing total time of one frame (1K data points).

5.2 Shared memory

During the implementation process, we experimented with using shared memory to accelerate calculations, given its significantly lower access time compared to other types of memory. Shared memory is particularly advantageous when multiple operations need to access the same data repeatedly. However, a key limitation of shared memory is that it is accessible only within a single block of threads. This restriction can limit its usefulness in broader applications where data needs to be shared across multiple block boundaries. In our approach, we aimed to keep the implementation straightforward by setting the number of threads per block to 512, matching the grid size. This setup ensures that all threads are organized into warps. There is also little or no difference in calculation time with a higher number of threads (Figure 11).

In this setup, each thread was responsible for copying its own grid cell value along with the values of neighboring cells above and below into shared memory. These values were then used in subsequent calculations. Despite the theoretical advantages of shared memory, our tests revealed that its use actually slow down the simulation as shown in Figure 13. This performance issue can be attributed to several factors. Primarily, the overhead associated with configuring and managing shared memory did not provide sufficient benefits to justify its use in our case. Additionally, since each cell's value was utilized no more than twice during computations, the benefit of rapid access provided by shared memory was diminished.

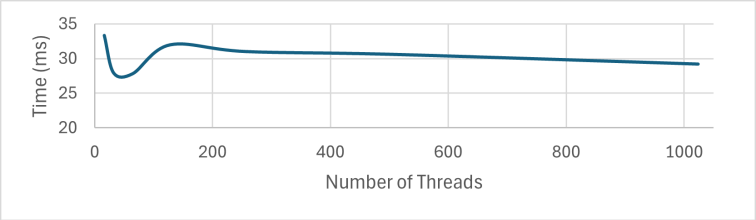


Figure 11: Total time of one frame for different threads per block (1K data points).

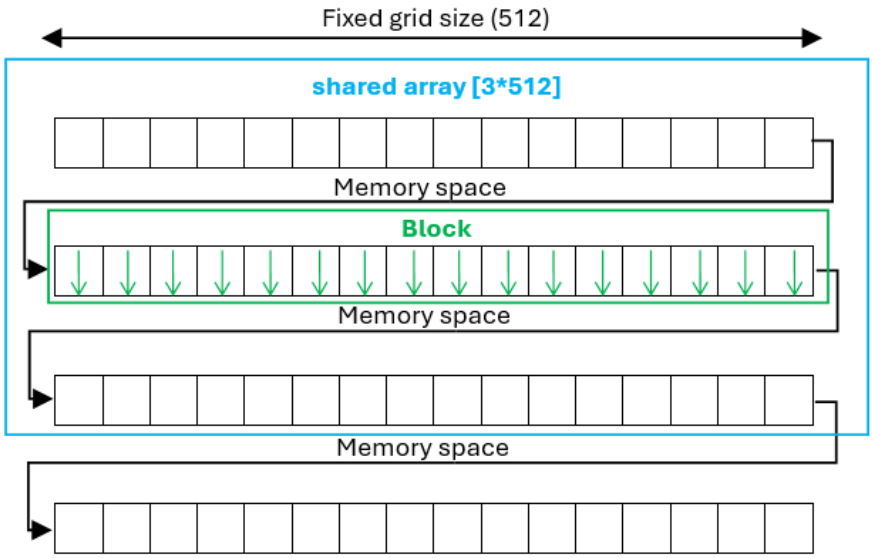


Figure 12: Layout of grid cells that get copied to shared memory of block.

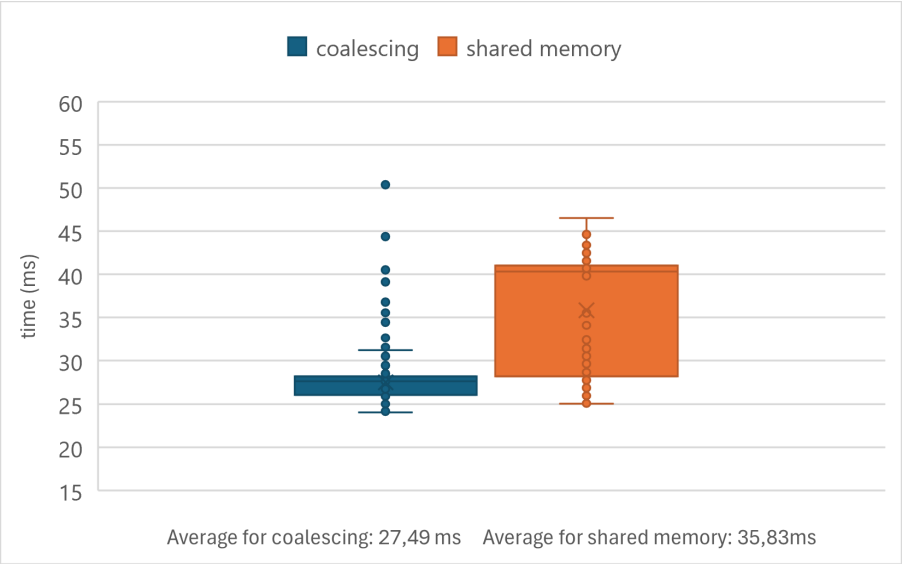


Figure 13: Coalescing vs shared memory total time of one frame (1K data points).

References

[1] J. Stam, "Real-time fluid dynamics for games," 2003. [Online]. Available: http://graphics.cs.cmu.edu/nsp/course/15-464/Fall09/papers/StamFluidforGames.pdf.

[2] Wikipedia contributors, "Gauss–seidel method — Wikipedia, the free encyclopedia." https://en.wikipedia.org/w/index.php?title=Gauss%E2%80%93Seidel_method&oldid=1222881727, 2024. [Online; accessed 8-May-2024].